



Test Automation Framework (TAF::Perl)

HammerDB TPROC-C Overview/Quick Start

MariaDB Foundation <https://mariadb.org>

Open, vendor-neutral, community-driven tooling for database testing and benchmarking.

TAF-Perl is maintained and published by the MariaDB Foundation to support transparent, reproducible, and fair performance evaluation across the open source database ecosystem. The framework is part of the Foundation's mission to provide open tooling that benefits the entire database community.

© MariaDB Foundation. All rights reserved.

HammerDB TPROC-C Overview

HammerDB TPROC-C is an open workload created and maintained by Steve Shaw. It models the general OLTP concepts described in the publicly available TPC-C specification from the Transaction Processing Performance Council.

The TPC-C specification can be found at: <https://www.tpc.org/tpcc/>

HammerDB is documented at: <https://www.hammerdb.com/>

Within TAF, the HammerDB TPROC-C test suite provides a deterministic and contributor-proof method for running these workloads across multiple database engines. TAF does not modify HammerDB, does not validate TPC-C compliance, and does not certify results. The suite focuses on lifecycle control, configuration merging, and consistent execution across environments.

The test suite is responsible for initialization, deterministic merging of default properties, user properties, and command-line overrides, execution of the supported test cases (flat, step, rampup, rampdown, mixed), and collection of results for downstream reporting. It enforces stable and reproducible behavior across repeated runs.

The suite does not implement HammerDB itself, does not manage database provisioning, and does not interpret or certify benchmark results. All workload semantics originate from HammerDB, and all OLTP concepts originate from the publicly available TPC-C specification.

By integrating HammerDB TPROC-C into TAF, the goal is to provide a consistent and script-driven method for running long-lived OLTP workloads across MariaDB, MySQL, PostgreSQL, and other engines. This ensures that performance testing remains transparent, repeatable, and aligned with the architectural principles of TAF.

TAF - Test Suite Structure and Locations

The HammerDB TPROC-C test suite follows the standard TAF layout. All components are located under the TAF installation root.

Test Suite Module: taf-perl/test_suites/hammerdb-tprocc.pm

This file defines lifecycle routines, configuration handling, and execution flow.

Default Properties: taf-perl/properties/default/hammerdb_tprocc_default.properties

Provides deterministic defaults for all supported databases. Users must not modify this file.

HammerDB Client Source: taf-perl/client_source/hammerdb/HammerDB-5.0/

Contains the HammerDB CLI executable, Tcl scripts, and agent binaries used by the test suite.

Script Directory: taf-perl/scripts/hammerdb/

Contains TAF wrapper scripts and helper logic used during execution.

Configuration Resolution Order

TAF resolves configuration in a strict and deterministic order. This guarantees that every test suite behaves the same way across environments and that users always know which value is in effect. All test suites, including HammerDB TPROC-C, follow the same precedence rules.

1. TAF and HammerDB TPROC-C Default Files

Both TAF and the HammerDB TPROC-C test suite provide default properties files. These files define safe, stable baseline settings and may be overridden, but must never be modified directly. Users override defaults only through user properties files.

2. Auto-Populated Values

When certain settings are left unset, TAF and the HammerDB TPROC-C test suite automatically populate them using installation paths or other environment-derived values. These values apply only when the user has not provided an explicit override.

3. User Properties Files

Users supply a test-case-specific properties file to override defaults. This is the intended method for running TAF. Any key defined in a user properties file replaces both the default value and any auto-populated value.

4. Command-Line Overrides

Command-line options have the highest precedence. Any key specified on the command line replaces the value from the user properties file, the auto-populated value, and the default.

This resolution order ensures predictable, contributor-proof behavior. The final configuration used by the test suite is always the result of applying these four layers in order, with command-line options taking priority over all other sources.

Test Suite Properties

Test suite properties begin with the prefix: **hammerdb_tprocc.<option>=**

These properties control workload behavior, client paths, agent settings, and database-specific extensions. Defaults come from the test suite's default properties file. Users may override any value through a user properties file or a command-line option.

Examples:

- hammerdb_tprocc.number_of_warehouses=20
- hammerdb_tprocc.checksum_after_setup=true
- hammerdb_tprocc.db_type=maria

TAF Options That Affect Test Suite Execution

The following is a sample of TAF options and properties that impact test suite execution. These options directly influence how the HammerDB TPROC-C test suite runs. Each option has both a command-line options form (O) and a properties form (P).

This is not a complete list. TAF provides many additional options and properties, all of which can be viewed by running:

perl taf.pl --help

Users are encouraged to review the full usage output to understand the complete set of capabilities.

Examples.

Action

O = --action=<action>

P = taf.action=<action>

Controls the overall TAF workflow.

Duration

O = --duration=<seconds|min>

P = taf.duration=<seconds|min>

How long to run each test.

Seconds vs. minutes depends entirely on the test suite implementation.

Default: 30

Iterations

O = --iterations=<number>

P = taf.iterations=<number>

Number of iterations to run for each thread count.

Default: 1

Thread Counts

O = --threads="2,4,..."

P = taf.threads="2,4,..."

Defines the list of thread counts to run.

These options combine with the test suite's own properties to determine the final workload behavior. All values are resolved using TAF's deterministic precedence rules: defaults, auto-populated values, user properties, then command-line overrides.

HammerDB-TPROCC Help

The following is example of running test suite help, and the output of the test suite's help.

perl taf.pl --test-suite=hammerdb-tprocc --list-test-suites-help

TAF TSM::LoadTestSuite -> : Called

TAF TSM::LoadTestSuite -> : Attempting to load: hammerdb-tprocc

TAF TSM::LoadTestSuite -> : Test suite path : /home/jeb/taf-perl/test_suites/hammerdb-tprocc.pm

TAF TSM::LoadTestSuite -> : Test suite has completed loading.

TAF TSM::LoadTestSuite -> : Complete

HammerDB TPROC-C Test Suite HELP

Default Tests:

tprocc

Legal Tests:

tprocc

Resolved Properties (final values in effect):

hammerdb_tprocc.agent = client_source/hammerdb/HammerDB-5.0/agent/agent
hammerdb_tprocc.agent_port = 10001
hammerdb_tprocc.agent_started_by_ts = 1
hammerdb_tprocc.allwarehouse = 0
hammerdb_tprocc.async_client = 10
hammerdb_tprocc.async_delay = 1000
hammerdb_tprocc.async_scale = 0
hammerdb_tprocc.async_verbose = 0
hammerdb_tprocc.checksum_after_run = 0
hammerdb_tprocc.checksum_after_setup = 0

```
hammerdb_tprocc.client_executable = client_source/hammerdb/HammerDB-5.0/hammerdbcli
hammerdb_tprocc.client_script_dir = scripts/hammerdb
hammerdb_tprocc.connect_pool = 0
hammerdb_tprocc.db_type = maria
hammerdb_tprocc.default_duration = 120
hammerdb_tprocc.default_rampup = 60
hammerdb_tprocc.driver = timed
hammerdb_tprocc.extra_args =
hammerdb_tprocc.include_html_metrics = 0
hammerdb_tprocc.include_html_result = 0
hammerdb_tprocc.include_html_tcount = 0
hammerdb_tprocc.include_html_timing = 0
hammerdb_tprocc.include_json_metrics = 0
hammerdb_tprocc.include_json_result = 0
hammerdb_tprocc.include_json_tcount = 0
hammerdb_tprocc.include_json_timing = 0
hammerdb_tprocc.keyandthink = 0
hammerdb_tprocc.log_to_temp = 0
hammerdb_tprocc.maria_history_pk = 0
hammerdb_tprocc.maria_no_stored_procs = 0
hammerdb_tprocc.maria_partition = 0
hammerdb_tprocc.maria_prepared = 0
hammerdb_tprocc.maria_purge = 0
hammerdb_tprocc.mssqls_bucket = 1
hammerdb_tprocc.mssqls_checkpoint = 0
hammerdb_tprocc.mssqls_durability = SCHEMA_AND_DATA
hammerdb_tprocc.mssqls_imdb = 0
hammerdb_tprocc.mssqls_use_bcp = 1
hammerdb_tprocc.mysql_history_pk = 0
hammerdb_tprocc.mysql_no_stored_procs = 0
hammerdb_tprocc.mysql_partition = 0
hammerdb_tprocc.mysql_prepared = 0
hammerdb_tprocc.number_of_warehouses = 10
hammerdb_tprocc.pg_cituscompat = 0
```

```
hammerdb_tprocc.pg_dritasnap = 0
hammerdb_tprocc.pg_oracompat = 0
hammerdb_tprocc.pg_partition = 0
hammerdb_tprocc.pg_storedprocs = 0
hammerdb_tprocc.pg_vacuum = 0
hammerdb_tprocc.raiseerror = 0
hammerdb_tprocc.show_out_put = 0
hammerdb_tprocc.test_client_version = HammerDB-5.0
hammerdb_tprocc.timeprofile = 1
hammerdb_tprocc.total_iterations = 100000000
```

TPROC-C Defaults and How to Modify Them:

The TPROC-C test suite loads its baseline defaults from:

```
properties/default/hammerdb_tprocc_default.properties
```

These defaults apply to ALL database types unless overridden by user-specified properties. The resolved values shown above are the final values after all overrides are applied.

To override a default, set:

```
hammerdb_tprocc.<option>=<value>
```

Example:

```
hammerdb_tprocc.allwarehouse=true
hammerdb_tprocc.async_client=20
```

TPROC-C Concepts:

TPROC-C models OLTP behavior using multiple virtual users executing transactions against a set of warehouses. The number of warehouses determines dataset size and scaling. Virtual users generate load; warehouses define the data.

Critical In-variants (Important):

1. Schema Build Rule:

During setup, builder threads must not exceed the warehouse count. HammerDB creates one builder per warehouse. Extra builders may idle or fail.

TAF enforces: threads = warehouse_count during setup.

2. Run Phase:

During workload execution, the requested thread count is used exactly as specified. No override is applied.

3. total_iterations:

Acts as a practical infinity when driver=timed. If set too low, the test ends early without warning.

4. MySQL Authentication Plugins:

HammerDB uses the system MySQL client library, which loads authentication plugins ONLY from the system plugin directory (typically /usr/lib64/mysql/plugin).

HammerDB does NOT honor plugin_dir overrides for TPROC-C. Users must ensure required plugins exist in the system directory when using MySQL-native-password.

```
#-----  
# MySQL Client Plugin Requirements for HammerDB CLI 5.0  
#-----
```

HammerDB CLI 5.0 uses the system libmysqlclient library. It does not load authentication plugins from any MySQL server installation directory, and it does not honor plugin_dir overrides for TPROC-C.

When connecting to older MySQL servers (8.0.x and 8.4.x),

the system client library may require two authentication plugins that are no longer shipped with those server versions:

1. mysql_native_password.so
2. auth_socket.so

MySQL 9.5 continues to ship these client-side authentication plugins for backward compatibility. Although the server-side native-password plugin was removed in MySQL 9.x, the client plugin remains fully compatible with older servers.

Placement Requirement:

Both .so files MUST be placed in the system plugin directory used by libmysqlclient. This is typically:

`/usr/lib64/mysql/plugin/`

HammerDB CLI 5.0 loads plugins ONLY from this directory. If these files are missing, authentication to MySQL 8.0 or 8.4 servers using native-password may fail.

Summary:

- libmysqlclient.so.24 from MySQL 9.5 can authenticate to MySQL 8.0 and 8.4 servers when the required client plugins are present.
- mysqld 9.x cannot use mysql_native_password on the server side, but the client plugin remains valid for benchmarking older servers with HammerDB CLI 5.0.

Common TPROC-C Options (All Databases):

driver:

Controls execution mode. Default is 'timed'.

total_iterations:

Upper bound on transactions per VU. If too low, test ends early without warning.

raiseerror:

When true, aborts VU on error. When false, logs only.

keyandthink:

Enables TPC-C think time. Disabled by default.

allwarehouse:

When true, enables cross-warehouse access. Reduces locality and lowers throughput.

timeprofile:

Enables per-VU profiling. Adds overhead at high VU counts.

async_client:

Number of async client threads. High values may overload the client host.

async_delay:

Delay (ms) between async operations.

async_verbose:

Verbose async logging. Large performance penalty.

connect_pool:

When false, each VU creates its own connection. High VU counts may exceed database connection limits.

Cross-Database Notes:

- All TPROC-C workload options are shared across all DBs.
- Only connection parameters differ by database type.
- `async_client` and `connect_pool` may behave differently depending on the database's connection model.
- `allwarehouse` affects locality differently across engines.

Helpful Sites:

<https://www.hammerdb.com>

<https://www.tpc.org/tpcc/>

<https://www.mariadb.org>

End of HammerDB TPROC-C Test Suite HELP

HammerDB-TPROCC Usage Command Example

Basic command-line for executing the provided example MariaDB user's properties-file.

`perl taf.pl --prop=./properties/mariadb/beta/hammerdb_tprocc_beta.properties`

User Properties File: `properties/mariadb/beta/hammerdb_tprocc_beta.properties`

```
#####
# TAF SETTINGS (GLOBAL)
#
# These settings control the behavior of TAF itself:
# - what action to perform
# - which report plugins to load
# - which database config file to use
# - how TAF connects to the database
#
# These settings apply to ALL test suites, not only HammerDB.
#####

# Action TAF should perform.
taf.action=init-start-db-run-tests

# Report plugins (text, csv, json, chart_html)
taf.report_plugin=taf_res_raw_text.pm,text.pm,chart_and_test_info_results_tables_html.pm
```

```

# Comment about why this run is being executed
taf.comments="Testing MariaDB using HammerDB TPROC-C test suite"

#####
# TEST EXECUTION SETTINGS
#####

# Duration of TPROC-C run in minutes
taf.duration=3

# Number of iterations per thread count
taf.iterations=3

# Test suite to load
taf.test_suite=hammerdb-tprocc

# Test type (adhoc, investigation, production, rerun, release)
taf.test_type=adhoc

# Test case from test suite (hammerdb-tprocc supports only tpcc)
taf.tests=tprocc

# Thread counts to run
taf.threads=10,20,30

# HammerDB warmup duration in minutes
taf.warmup_duration=1

# Sleep X seconds after test setup.
taf.sleep_after_test_setup=60

# Sleep X seconds after test run.
taf.sleep_after_test_run=60

#####
# DATABASE SETTINGS
#
# These settings control how TAF connects to the MariaDB server for this run.
# Any value left commented out will fall back to:
# 1. TAF defaults
# 2. auto-populated values (special null handling)
# 3. command-line overrides
#####

# Path to the MariaDB configuration file to feed into mariadb.
# This must be a valid MariaDB config file.
taf.db_config_file=./database_config_files/mariadb/mariadb_simple_2gbp.cnf

# Optional: directory where the MariaDB software has been unpacked.
# If omitted, TAF attempts to auto-detect the install directory.
# taf.db_software_install_dir=/home/jeb/taf-perl/database_software_installs/mariadb-
12.3.0-linux-systemd-x86_64/

# Whether to connect using a UNIX socket instead of TCP.

```

```

# true = use socket
# false = use TCP (default)
taf.db_clients_use_unix_socket=true

# Optional: explicit socket path for MySQL connections.
# Only needed when using UNIX socket mode.
# taf.db_socket=/home/jeb/taf-perl/tmp/mariadb.sock

# # Optional overrides for database connection parameters.
# Uncomment only if you need to override TAF defaults.
# taf.database=test
# taf.engine=INNODB
# taf.db_user_pass=MariadbPass_@123
# taf.db_user=mariadb_tester
# taf.host=localhost
# taf.db_socket=/home/jeb/taf-perl/tmp/mysql.sock

#####
# HAMMERDB-TPROCC TEST SUITE SETTINGS
#
# These settings apply ONLY to the HammerDB-TPROCC test suite.
# They control:
# - warehouse count
# - database type for TCL generation
# - checksum behavior
# - which HammerDB reports to include
#
# These override the defaults in hammerdb_tprocc_default.properties.
#####

# Number of warehouses to create and load
hammerdb_tprocc.number_of_warehouses=10

# Database type for TCL configuration
hammerdb_tprocc.db_type=maria

# Checksum after setup
hammerdb_tprocc.checksum_after_setup=true

# Checksum after each run
hammerdb_tprocc.checksum_after_run=true

# HammerDB reports per iteration
# Reports are found in iterations sub results directory

# JSON report options
hammerdb_tprocc.include_json_tcount=true
hammerdb_tprocc.include_json_timing=true
hammerdb_tprocc.include_json_result=true
hammerdb_tprocc.include_json_metrics=true

# HTML report options
hammerdb_tprocc.include_html_tcount=true
hammerdb_tprocc.include_html_timing=true
hammerdb_tprocc.include_html_result=true
hammerdb_tprocc.include_html_metrics=true

```

```
##### TAF Debug #####  
taf.tools_debug=true  
taf.verbose=true
```

End of Document

This Quick Start was prepared and published by the MariaDB Foundation <https://mariadb.org>

TAF::Perl is part of the Foundation's commitment to open, vendor-neutral tooling for the database ecosystem. The framework is maintained to support fair benchmarking, reproducible testing, and transparent engineering practices across all database makers.

For contributions, feedback, or questions, please visit the Foundation website or contact the maintainers directly.

© MariaDB Foundation. All rights reserved.